

Chapter 3: Optimization

Basics and Applications of Deep Learning

Fall 2026

Department of Mathematical Data Science

Hanyang University ERICA

Lecturer: Sehyun Kwon



Optimization

So far,

- We compute the loss $L(W)$ using the training data.
- We then adjust the weights W to minimize the loss.

I.e., we want to find $W^* = \operatorname{argmin}_W L(W)$

Optimization

So far,

- We compute the loss $L(W)$ using the training data.
- We then adjust the weights W to minimize the loss.

I.e., we want to find $W^* = \operatorname{argmin}_W L(W)$

How can we find the W^* ?

Strategy #1: A first very bad idea solution: Random search

Random Search

$L_{\text{best}} \leftarrow \infty$

for $t = 1, 2, \dots, T$ **do**

$W_t \leftarrow$ randomly sample weights

$L_t \leftarrow L(W_t)$

if $L_t < L_{\text{best}}$ **then**

$W^* \leftarrow W_t$

$L_{\text{best}} \leftarrow L_t$

end if

end for

return W^*

Strategy #1: A first very bad idea solution: Random search

Random Search

$L_{\text{best}} \leftarrow \infty$

for $t = 1, 2, \dots, T$ **do**

$W_t \leftarrow$ randomly sample weights

$L_t \leftarrow L(W_t)$

if $L_t < L_{\text{best}}$ **then**

$W^* \leftarrow W_t$

$L_{\text{best}} \leftarrow L_t$

end if

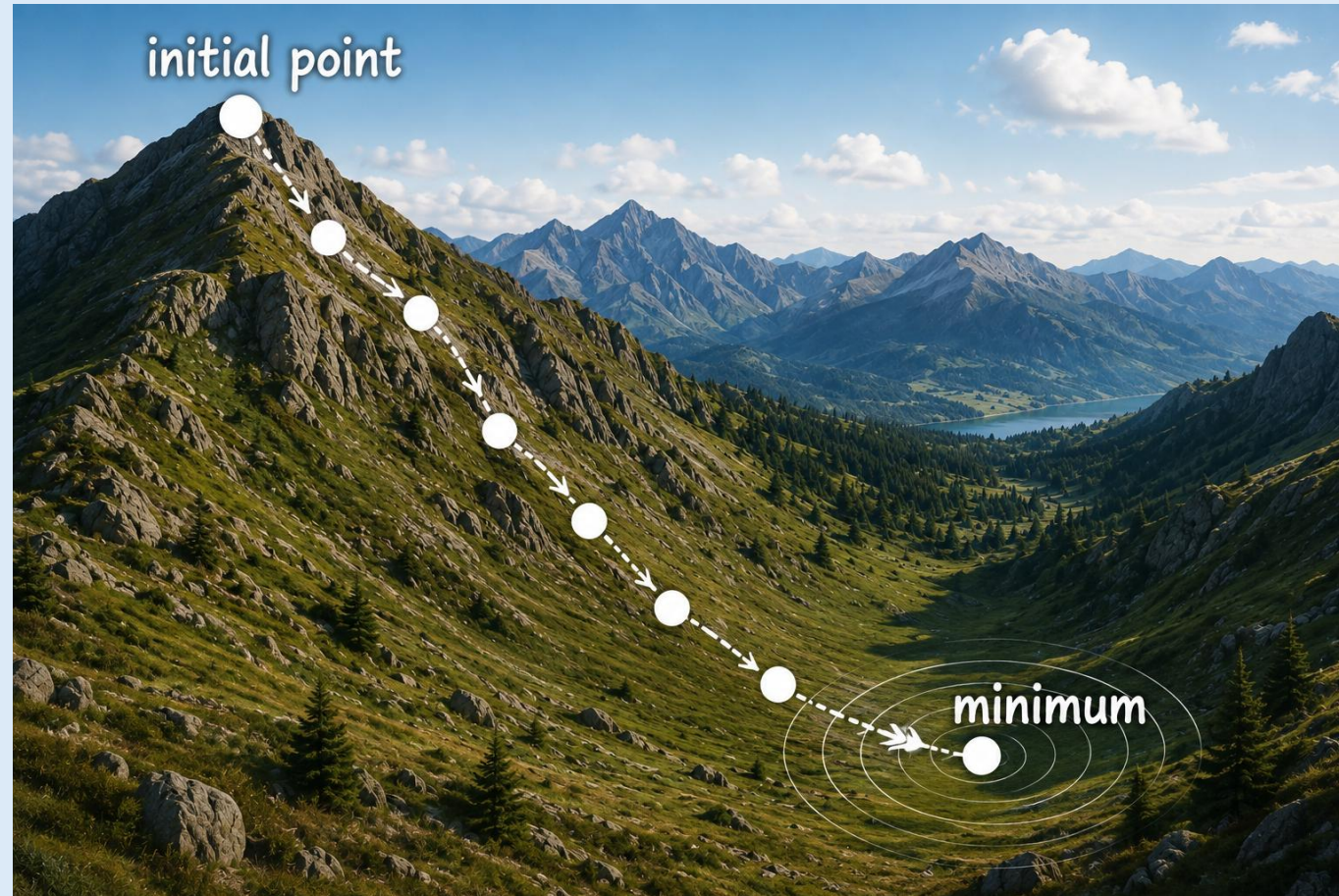
end for

return W^*

In a high-dimensional weight space, good solutions occupy only a tiny region.

Random sampling is therefore very unlikely to find one efficiently.

Strategy #2: Follow the slope



Strategy #2: Follow the slope

In 1-dimension, the derivative of a function:

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Strategy #2: Follow the slope

In 1-dimension, the derivative of a function: $\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$

In multi-dimensions, the **gradient** is the vector of along each dimension:

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]$$

Strategy #2: Follow the slope

In 1-dimension, the derivative of a function: $\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$

In multi-dimensions, the **gradient** is the vector of along each dimension:

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]$$

The slope in any direction is the **dot product** of the direction with the gradient:

$$D_{\mathbf{u}}f(\mathbf{x}) = \nabla f(\mathbf{x})^T \mathbf{u}$$

Strategy #2: Follow the slope

In 1-dimension, the derivative of a function: $\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$

In multi-dimensions, the **gradient** is the vector of along each dimension:

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]$$

The slope in any direction is the **dot product** of the direction with the gradient:

$$D_{\mathbf{u}}f(\mathbf{x}) = \nabla f(\mathbf{x})^T \mathbf{u}$$

The direction of steepest descent is the **negative gradient**:

$$\mathbf{u}_{\text{steepest}} = -\frac{\nabla f(\mathbf{x})}{\|\nabla f(\mathbf{x})\|}$$

current W

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33, ...]

loss 1.25347

gradient dW

[?,
?,
?,
?,
?,
?,
?,
?,
?, ...]

current W

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33, ...]

loss 1.25347

W + h (first dim)

[0.34 + **0.0001**,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33, ...]

loss 1.25322

gradient dW

[?,
?,
?,
?,
?,
?,
?,
?,
?, ...]

current W

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33, ...]

loss 1.25347

W + h (first dim)

[0.34 + **0.0001**,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33, ...]

loss 1.25322

gradient dW

[-2.5,
?,
?,

$$(1.25322 - 1.25347)/0.0001 = -2.5$$

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

?, ...]

current W

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33, ...]

loss 1.25347

W + h (first dim)

[0.34,
-1.11 + **0.0001**,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33, ...]

loss 1.25353

gradient dW

[-2.5,
?,
?,
?,
?,
?,
?,
?,
?, ...]

current W

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33, ...]

loss 1.25347

W + h (first dim)

[0.34,
-1.11 + **0.0001**,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33, ...]

loss 1.25353

gradient dW

[-2.5,
0.6,
?,
?, ...]

$$(1.25353 - 1.25347)/0.0001 = 0.6$$

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

?, ...]

current W

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33, ...]

loss 1.25347

W + h (first dim)

[0.34,
-1.11,
0.78 + **0.0001**,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33, ...]

loss 1.25347

gradient dW

[-2.5,
0.6,
?,
?,
?,
?,
?,
?,
?, ...]

current W

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33, ...]

loss 1.25347

W + h (first dim)

[0.34,
-1.11,
0.78 + **0.0001**,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33, ...]

loss 1.25347

gradient dW

[-2.5,
0.6,
0,
?,


$$(1.25347 - 1.25347)/0.0001 = 0$$

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

current W

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33, ...]

loss 1.25347

W + h (first dim)

[0.34,
-1.11,
0.78 + **0.0001**,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33, ...]

loss 1.25347

gradient dW

[-2.5,
0.6,
0,
?,

Numeric Gradient

- Slow! Need to loop over all dimensions
- Approximate

?, ...]

This is inefficient. The loss is just a function of W .

$$L = \frac{1}{N} \sum_{i=1}^N L_i + \sum_k W_k^2$$

$$L_i = -\log \left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}} \right)$$

$$s = f(x; W) = Wx$$

want $\nabla_W L$

Use calculus to compute an
analytic gradient!



Isaac Newton
(1643-1727)



Gottfried Wilhelm
Leibniz
(1646-1716)

current W

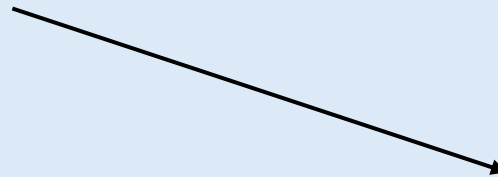
[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33, ...]

loss 1.25347

gradient dW

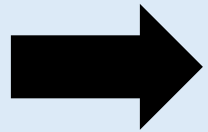
[-2.5,
0.6,
0,
0.2,
0.7,
-0.5,
1.1,
1.3,
-2.1, ...]

$dW = \dots$
(some function of
data and W)



In summary:

- Numerical gradient: approximate, slow, easy to write
- Analytic gradient: exact, fast, error-prone



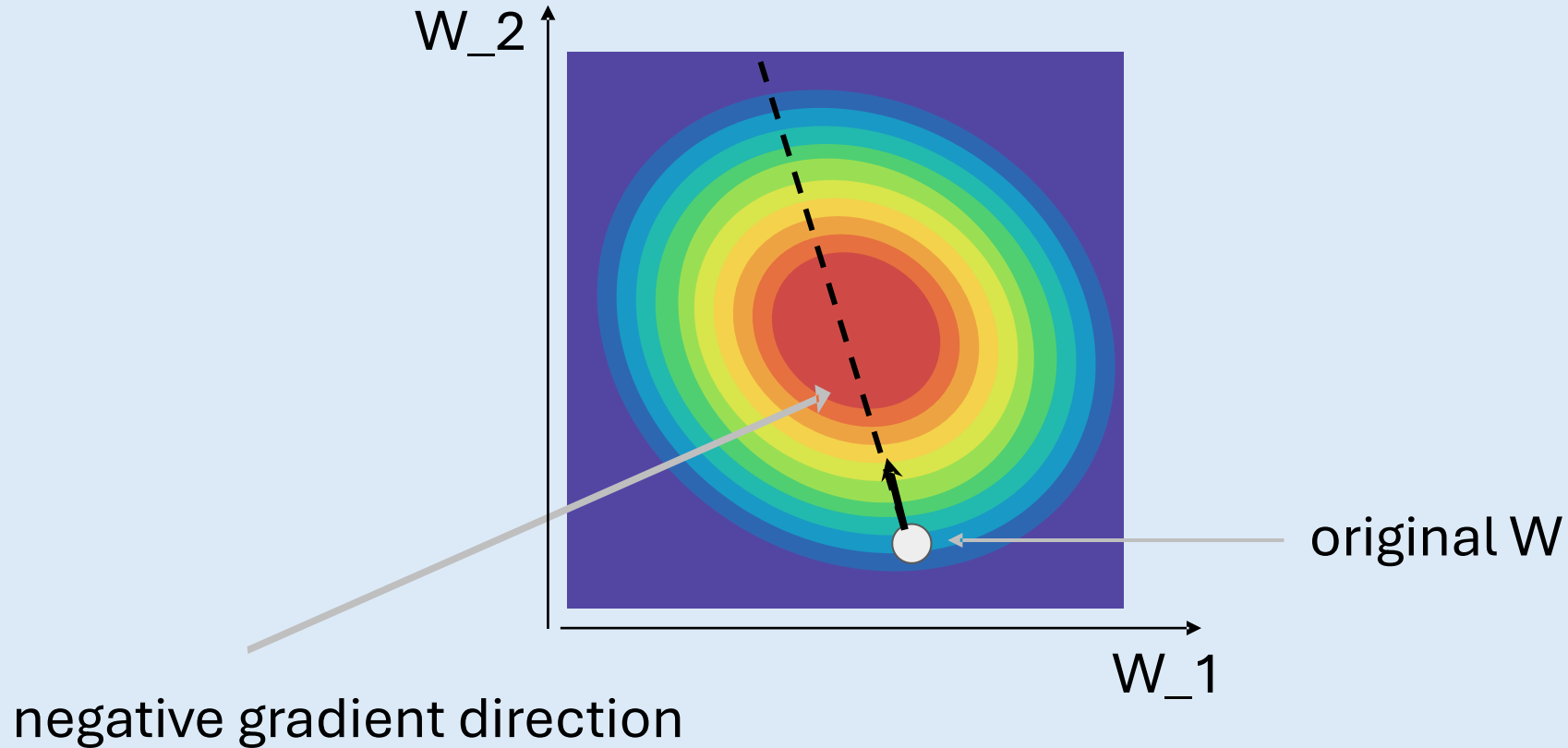
In practice: Always use analytic gradient (in this class), but check implementation with numerical gradient. This is called a gradient check.

Gradient Descent (GD)

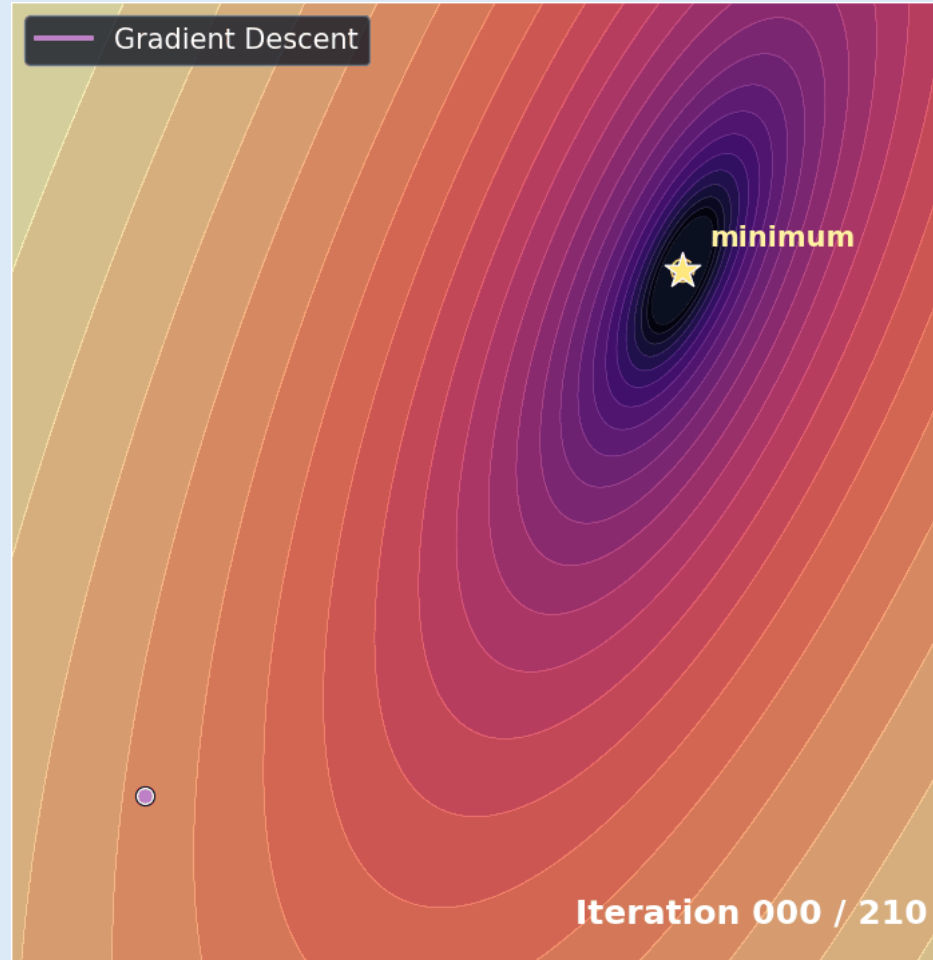
```
# Vanilla Gradient Descent

while True:
    weights_grad = evaluate_gradient(loss_func, data, weights)
    weights += -step_size * weights_grad # parameter update
```

Gradient Descent (GD)



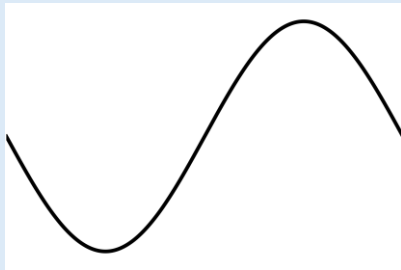
Gradient Descent (GD)



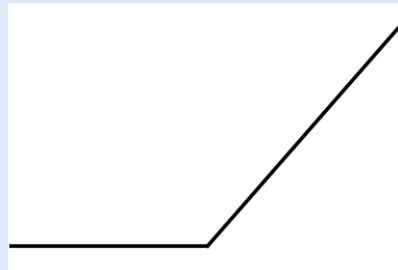
Definition of “differentiability”

In math, a function is “differentiable” if its derivative exists everywhere.

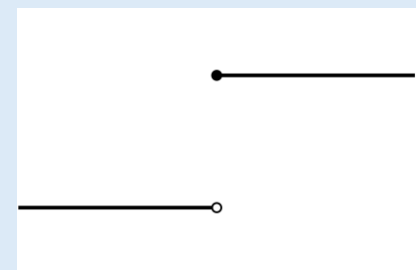
In deep learning, a function is often considered differentiable if its derivative exists everywhere except at a finite number of points and the function is nice.



*Differentiable in
DL & Math*



*Differentiable in
DL but not in Math*



*Not Differentiable
in DL or Math*

Stochastic Gradient Descent (SGD)

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(x_i, y_i, W) + \lambda R(W)$$

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W)$$

Full sum expensive
when **N** is large!

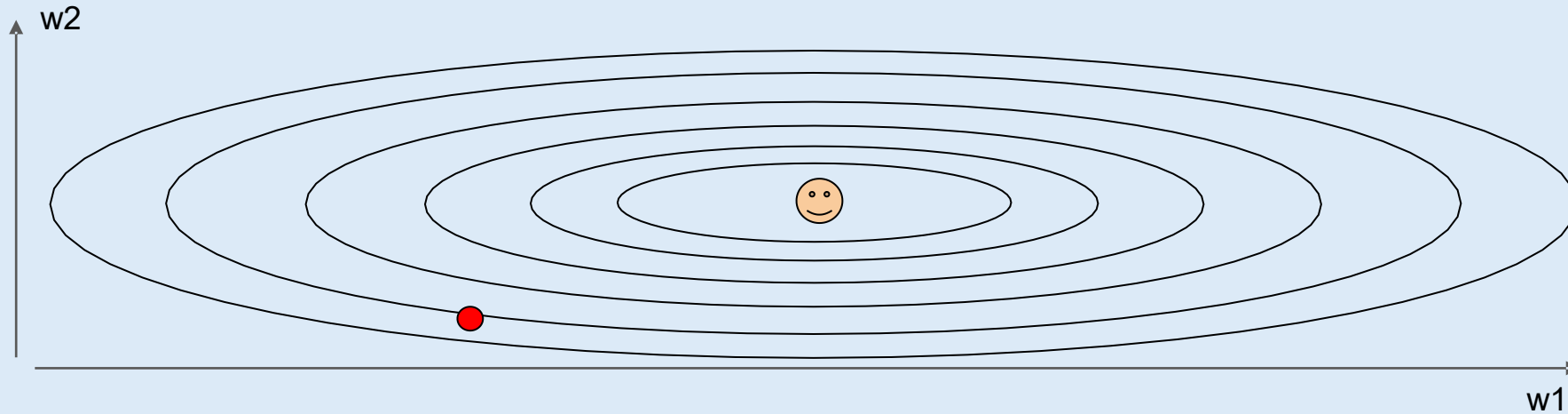
Approximate sum
using a **minibatch** of
examples
32 / 64 / 128 common

```
# Vanilla Minibatch Gradient Descent

while True:
    data_batch = sample_training_data(data, 256) # sample 256 examples
    weights_grad = evaluate_gradient(loss_fun, data_batch, weights)
    weights += -step_size * weights_grad # parameter update
```

Optimization: Problem #1 with SGD

What if loss changes quickly in one direction and slowly in another?
What does gradient descent do?

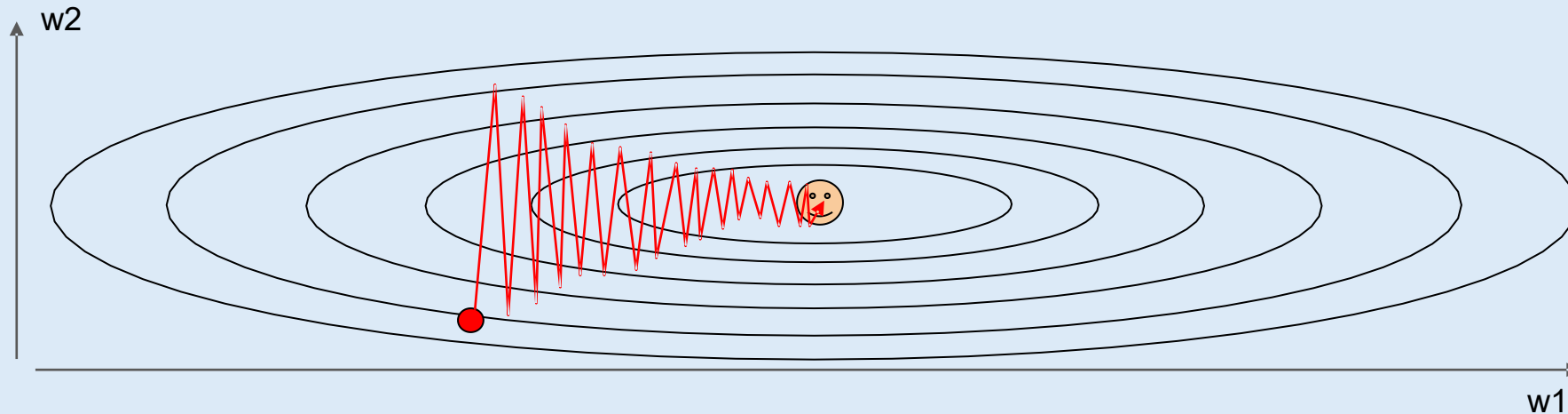


Optimization: Problem #1 with SGD

What if loss changes quickly in one direction and slowly in another?

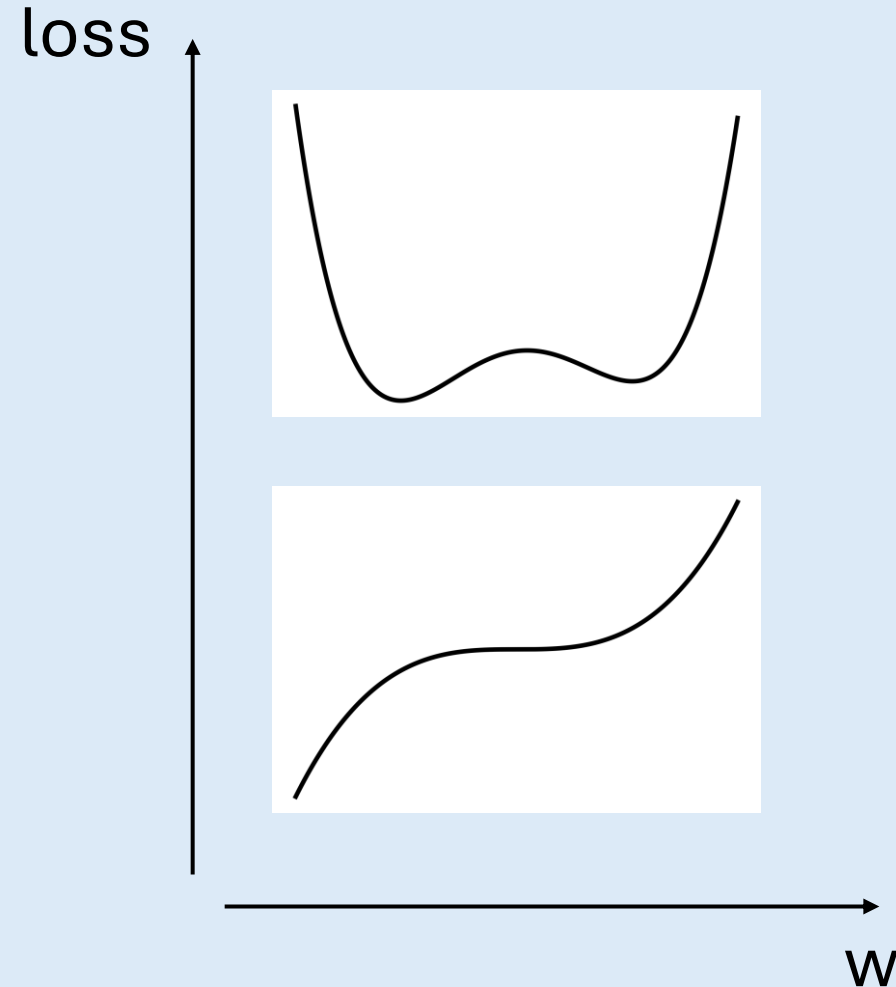
What does gradient descent do?

Very slow progress along shallow dimension, jitter along steep direction



Optimization: Problem #2 with SGD

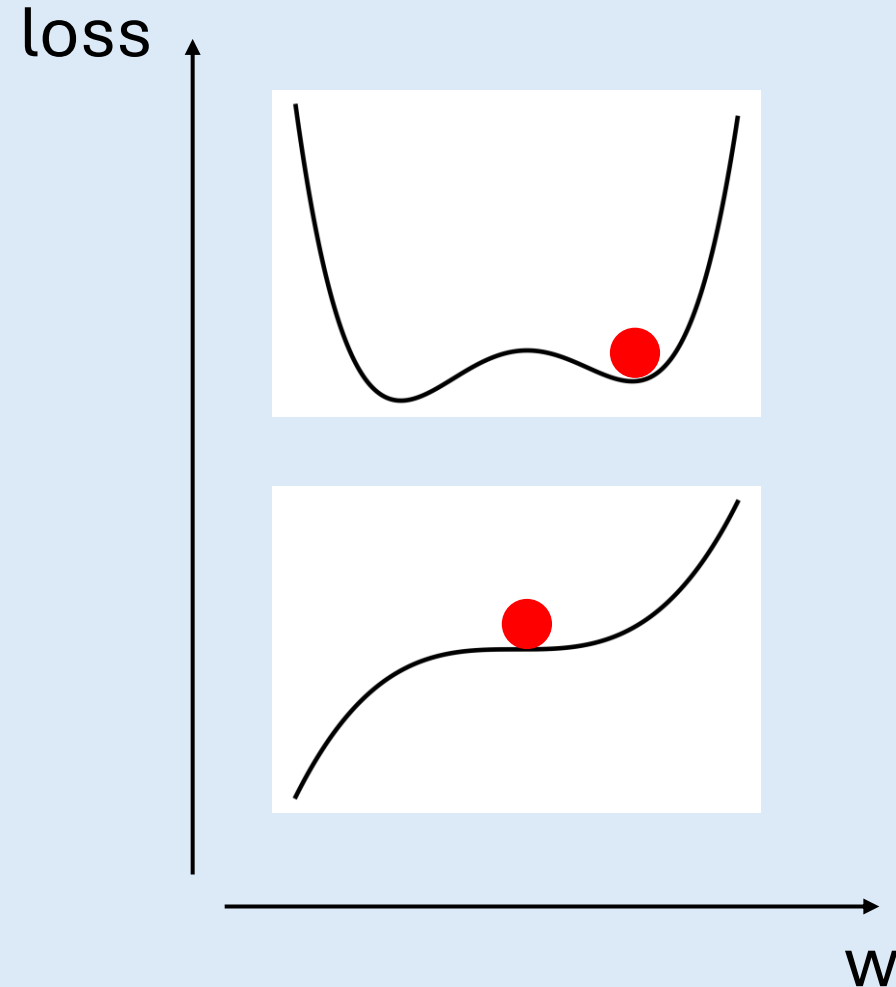
What if the loss function has a **local minima** or **saddle point**?



Optimization: Problem #2 with SGD

What if the loss function has a **local minima** or **saddle point**?

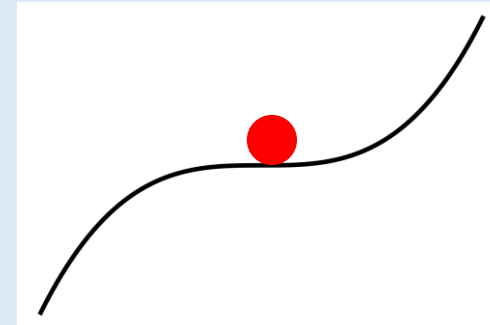
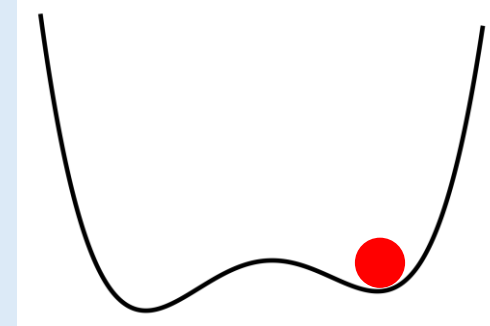
Zero gradient,
-> gradient descent gets stuck



Optimization: Problem #2 with SGD

What if the loss function has a **local minima** or **saddle point**?

Saddle points much more common in high dimension



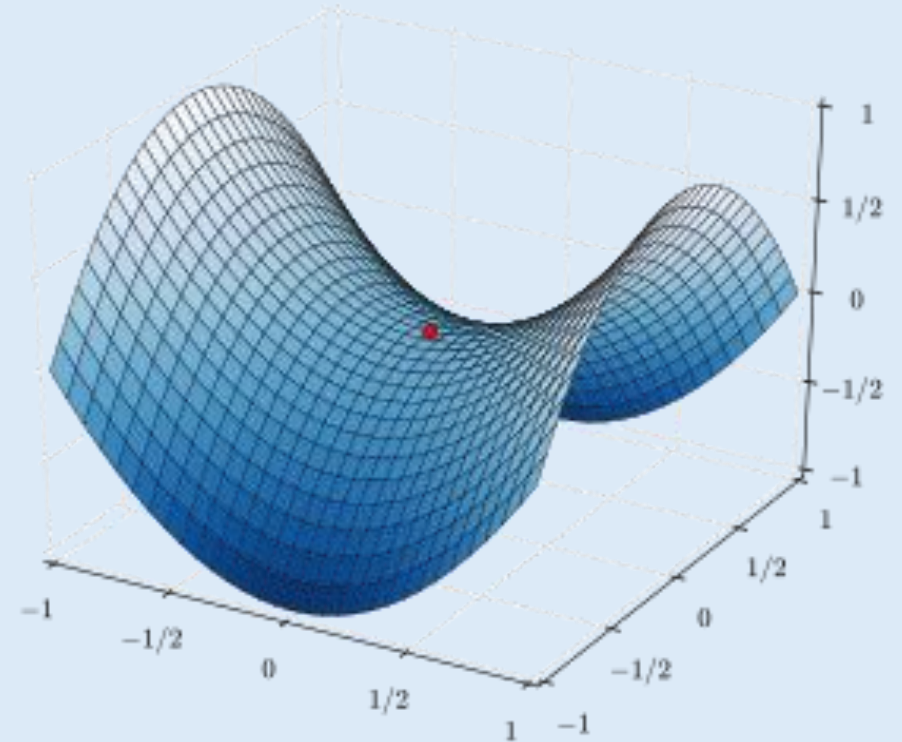
Optimization: Problem #2 with SGD

saddle point in two dimension

$$f(x, y) = x^2 - y^2$$

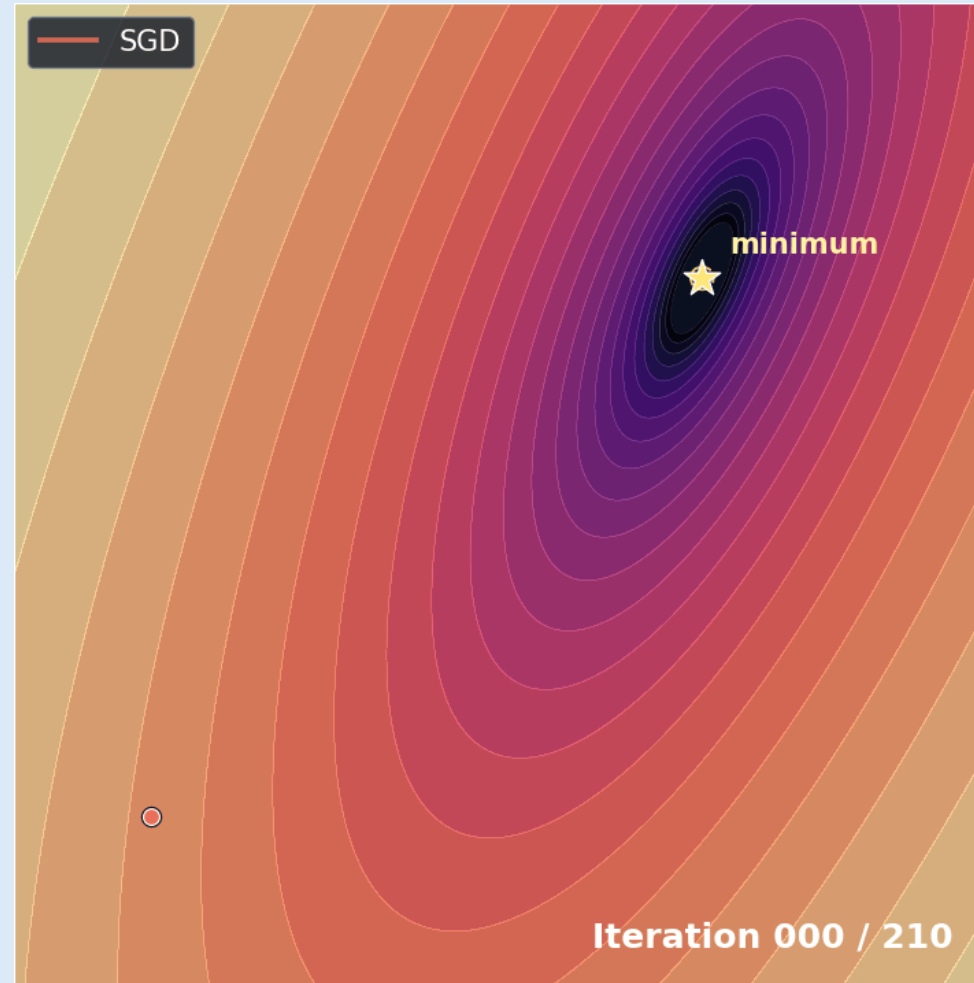
$$\frac{\partial}{\partial x} (x^2 - y^2) = 2x \rightarrow 2(0) = 0$$

$$\frac{\partial}{\partial y} (x^2 - y^2) = -2y \rightarrow -2(0) = 0$$



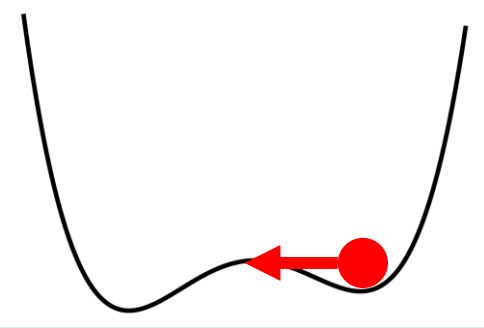
Optimization: Problem #3 with SGD

Because SGD uses a mini-batch gradient as an estimator of the full gradient, the estimated gradient contains error and is therefore noisy.

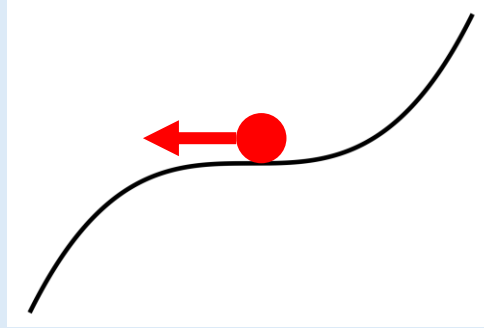


SGD + Momentum

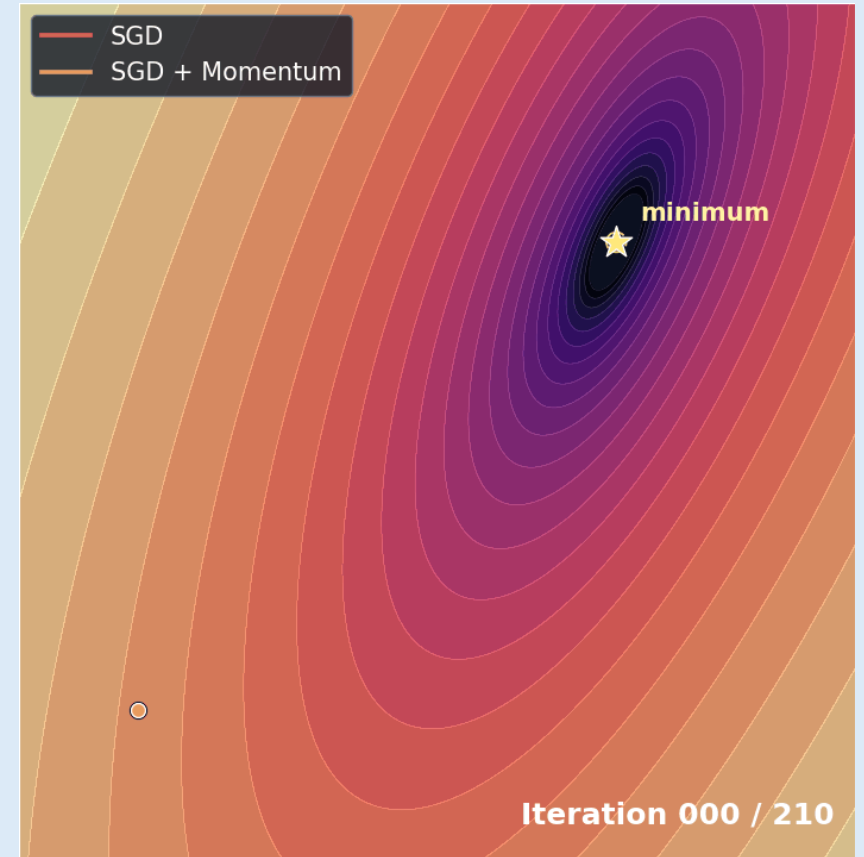
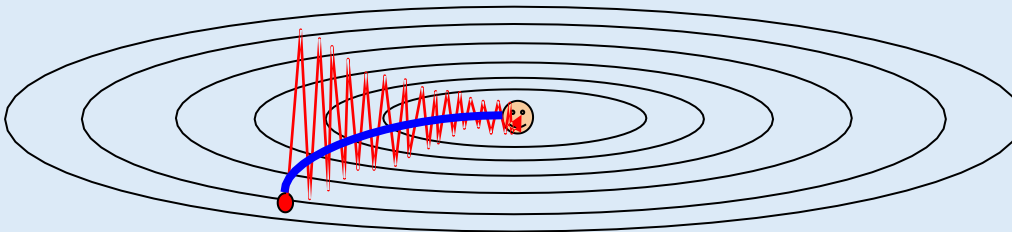
local minima



Saddle points



Poor conditioning



SGD: the simple two line update code

SGD

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:  
    dx = compute_gradient(x)  
    x -= learning_rate * dx
```

SGD: the simple two line update code

SGD

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
    dx = compute_gradient(x)
    x -= learning_rate * dx
```

SGD+Momentum

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

$$x_{t+1} = x_t - \alpha v_{t+1}$$

- Build up "velocity" as a running mean of gradients
- ρ gives "friction"; typically $\rho = 0.9$ or 0.99

SGD: the simple two line update code

SGD

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
    dx = compute_gradient(x)
    x -= learning_rate * dx
```

SGD+Momentum

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

$$x_{t+1} = x_t - \alpha v_{t+1}$$

```
vx = 0
while True:
    dx = compute_gradient(x)
    vx = rho * vx + dx
    x -= learning_rate * vx
```

- Build up "velocity" as a running mean of gradients
- ρ gives "friction"; typically $\rho = 0.9$ or 0.99

SGD: the simple two line update code

SGD

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
    dx = compute_gradient(x)
    x -= learning_rate * dx
```

SGD+Momentum

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

$$x_{t+1} = x_t - \alpha v_{t+1}$$

```
vx = 0
while True:
    dx = compute_gradient(x)
    vx = rho * vx + dx
    x -= learning_rate * vx
```

You may see SGD+Momentum formulated different ways,
but they are equivalent - give same sequence of x

More complex optimizers: RMSProp

**SGD +
Momentum**

```
vx = 0
while True:
    dx = compute_gradient(x)
    vx = rho * vx + dx
    x -= learning_rate * vx
```

Adds element-wise scaling of the gradient based on the historical sum of squares in each dimension (with decay)

RMSProp

```
grad_squared = 0
while True:
    dx = compute_gradient(x)
    grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dx * dx
    x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

More complex optimizers: RMSProp

**SGD +
Momentum**

```
vx = 0
while True:
    dx = compute_gradient(x)
    vx = rho * vx + dx
    x -= learning_rate * vx
```

"Per-parameter learning rates" or
"adaptive learning rates"

RMSProp

```
grad_squared = 0
while True:
    dx = compute_gradient(x)
    grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dx * dx
    x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```


RMSProp

RMSProp

```
grad_squared = 0
while True:
    dx = compute_gradient(x)
    grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dx * dx
    x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

Q: What happens with RMSProp?

RMSProp

RMSProp

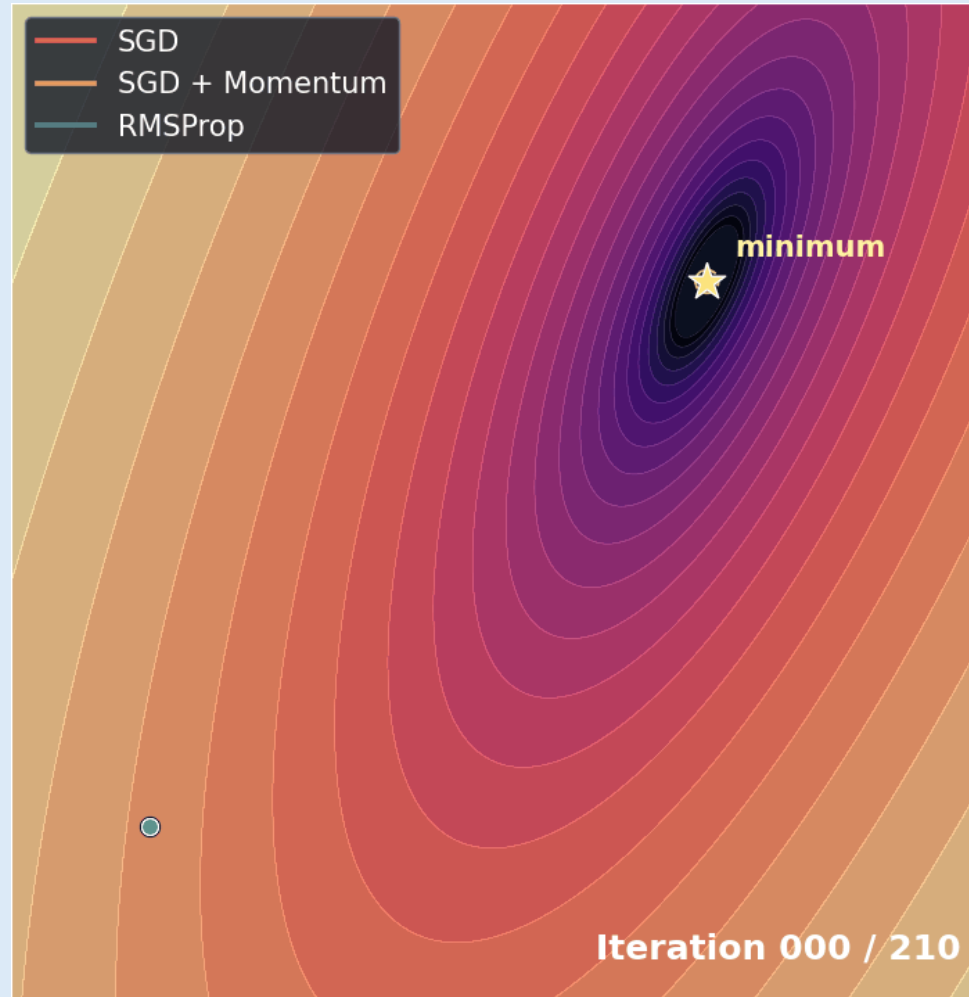
```
grad_squared = 0
while True:
    dx = compute_gradient(x)
    grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dx * dx
    x -= learning_rate * dx / (np.sqrt(grad_squared) + 1e-7)
```

Normalized term!

Q: What happens with RMSProp?

- Progress along "steep" directions is damped
- Progress along "flat" directions is accelerated

RMSProp



Optimizers: Adam (almost)

```
first_moment = 0
second_moment = 0
while True:
    dx = compute_gradient(x)
    first_moment = beta1 * first_moment + (1 - beta1) * dx
    second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
    x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7)
```

Adam (almost)

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
    dx = compute_gradient(x)
    first_moment = beta1 * first_moment + (1 - beta1) * dx
    second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
    x -= learning_rate * first_moment / (np.sqrt(second_moment) + 1e-7)
```

Momentum

RMSProp

Sort of like RMSProp with momentum

Q: What happens at first timestep?

Adam (full form)

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
    dx = compute_gradient(x)
    first_moment = beta1 * first_moment + (1 - beta1) * dx
    second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
    first_unbias = first_moment / (1 - beta1 ** t)
    second_unbias = second_moment / (1 - beta2 ** t)
    x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7)
```

Momentum

Bias correction

AdaGrad / RMSProp

Bias correction for the fact that
first and second moment
estimates start at zero

Adam (full form)

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
    dx = compute_gradient(x)
    first_moment = beta1 * first_moment + (1 - beta1) * dx
    second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
    first_unbias = first_moment / (1 - beta1 ** t)
    second_unbias = second_moment / (1 - beta2 ** t)
    x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7)
```

Momentum

Bias correction

AdaGrad / RMSProp

Bias correction for the fact that first and second moment estimates start at zero

Adam with $\text{beta1} = 0.9$, $\text{beta2} = 0.999$, and $\text{learning_rate} = 1\text{e-}3$ or $5\text{e-}4$ is a great starting point for many models!

Adam (full form)

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
    dx = compute_gradient(x)
    first_moment = beta1 * first_moment + (1 - beta1) * dx
    second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
    first_unbias = first_moment / (1 - beta1 ** t)
    second_unbias = second_moment / (1 - beta2 ** t)
    x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7)
```

Momentum

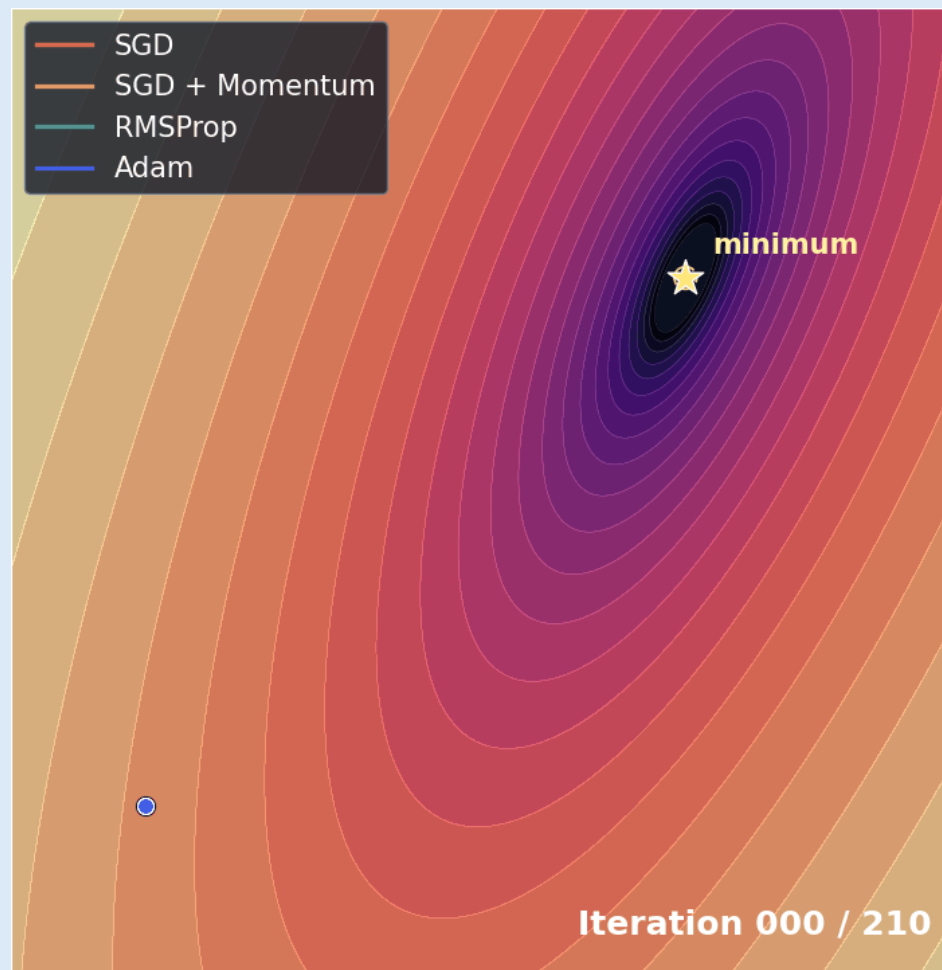
Bias correction

AdaGrad / RMSProp

Bias correction for the fact that first and second moment estimates start at zero

Adam with $\text{beta1} = 0.9$, $\text{beta2} = 0.999$, and $\text{learning_rate} = 1\text{e-}3$ or $5\text{e-}4$ is a great starting point for many models!

Adam



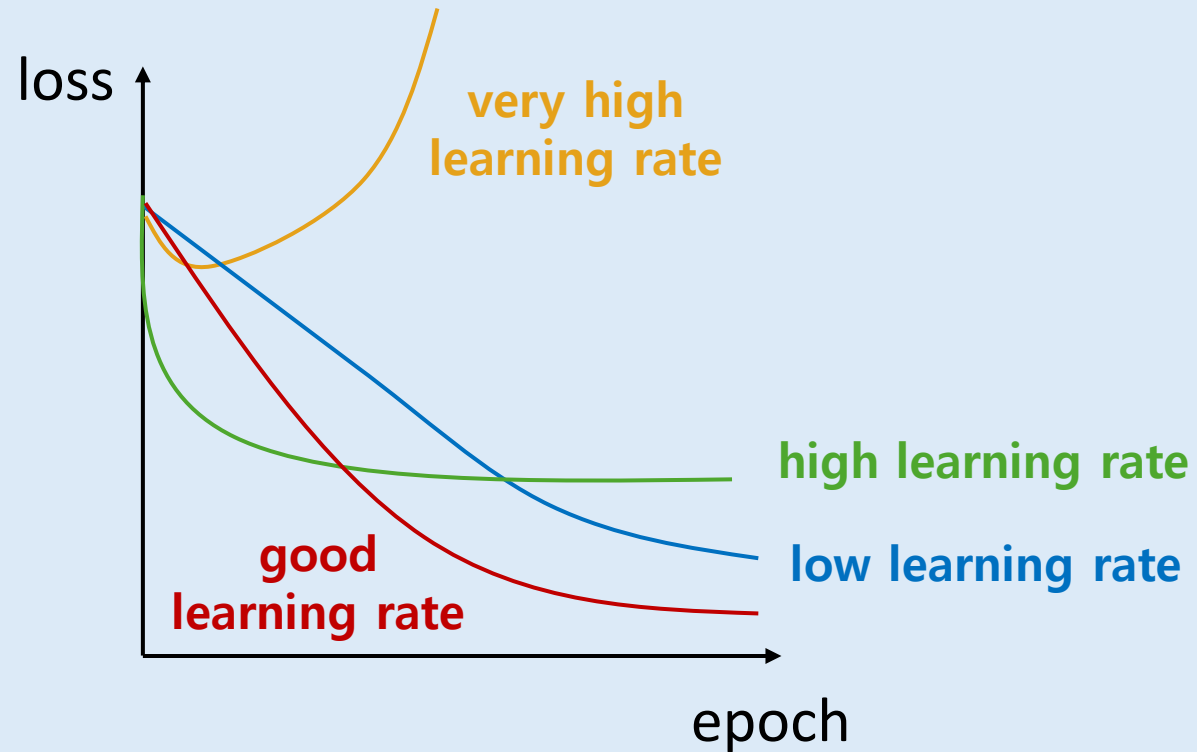
Learning rate schedules

```
# Vanilla Gradient Descent
while True:
    weights_grad = evaluate_gradient(loss_fun, data, weights)
    weights += - step_size * weights_grad # parameter update
```

↓
Learning rate

Learning rate schedules

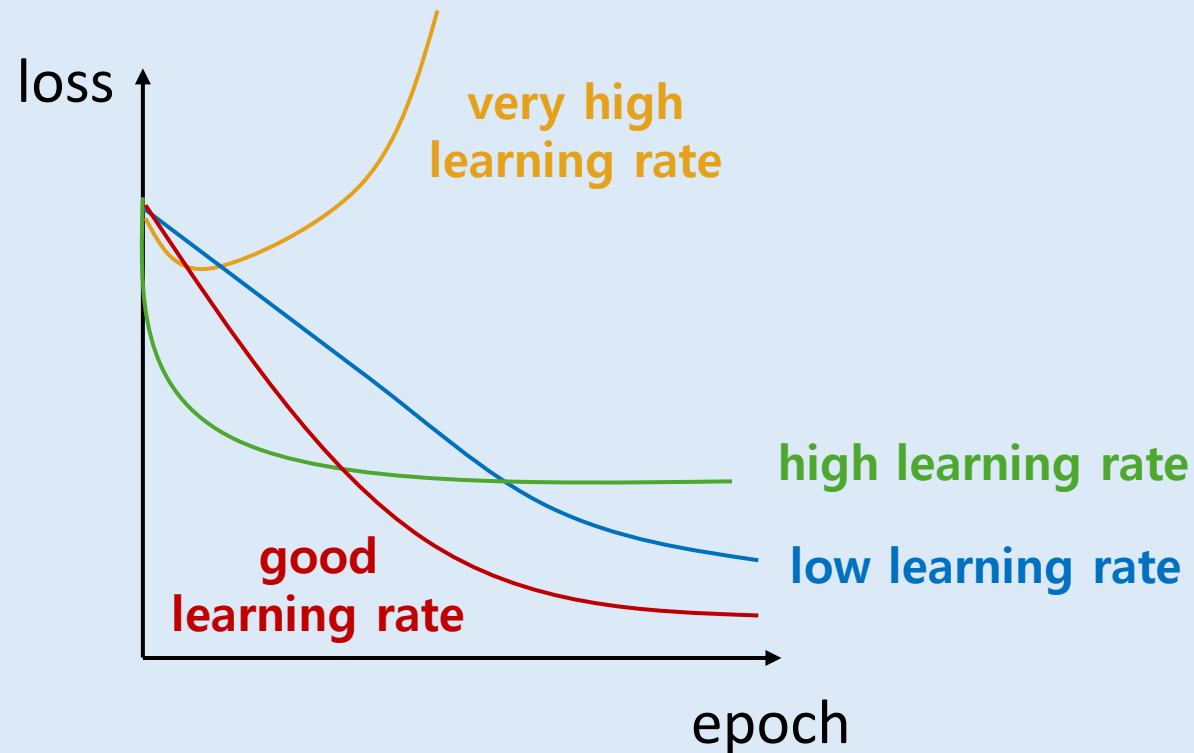
SGD, SGD+Momentum, RMSProp, Adam all have **learning rate** as a hyperparameter.



Q: Which one of these learning rates is best to use?

Learning rate schedules

SGD, SGD+Momentum, RMSProp, Adam all have **learning rate** as a hyperparameter.

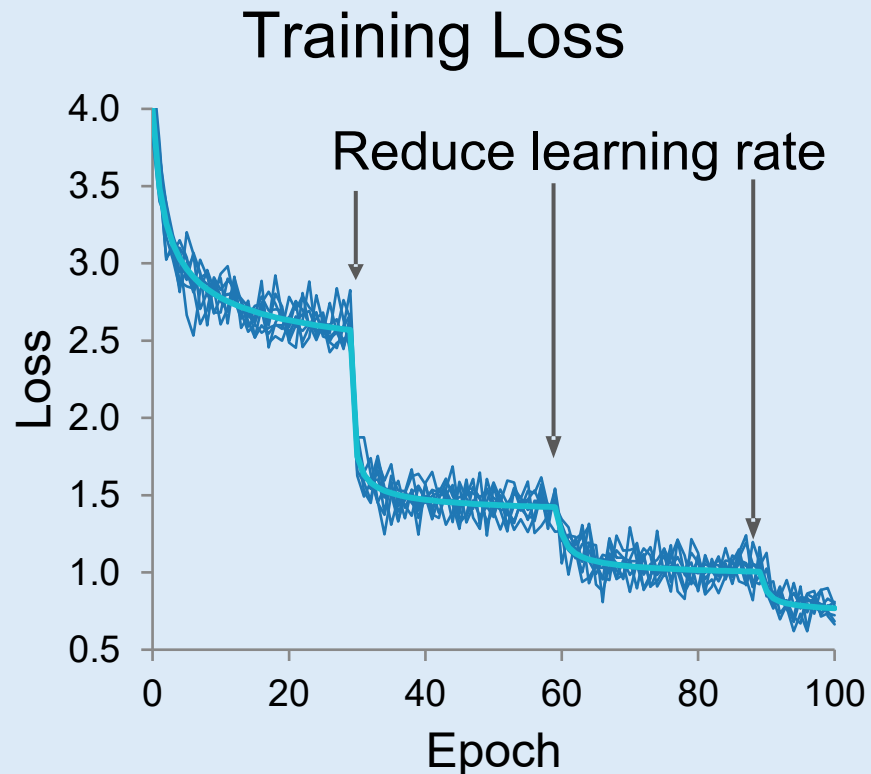


Q: Which one of these learning rates is best to use?

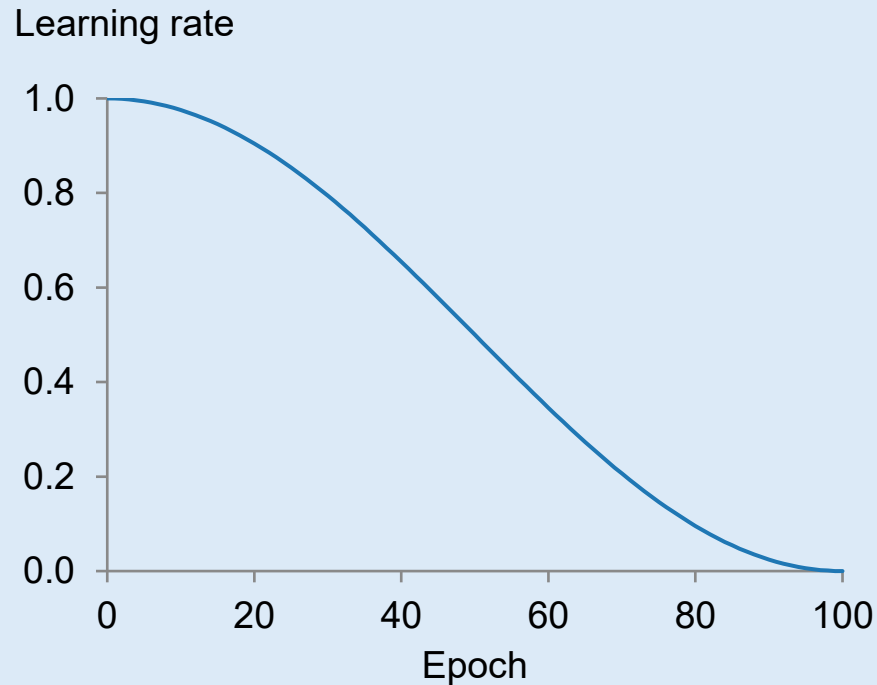
A: In reality, all of these could be good learning rates.

Learning rate decays over time

Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.



Learning rate decay



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine:
$$\alpha_t = \frac{1}{2}\alpha_0 \left(1 + \cos \left(\frac{t\pi}{T} \right) \right)$$

α_0 : Initial learning rate

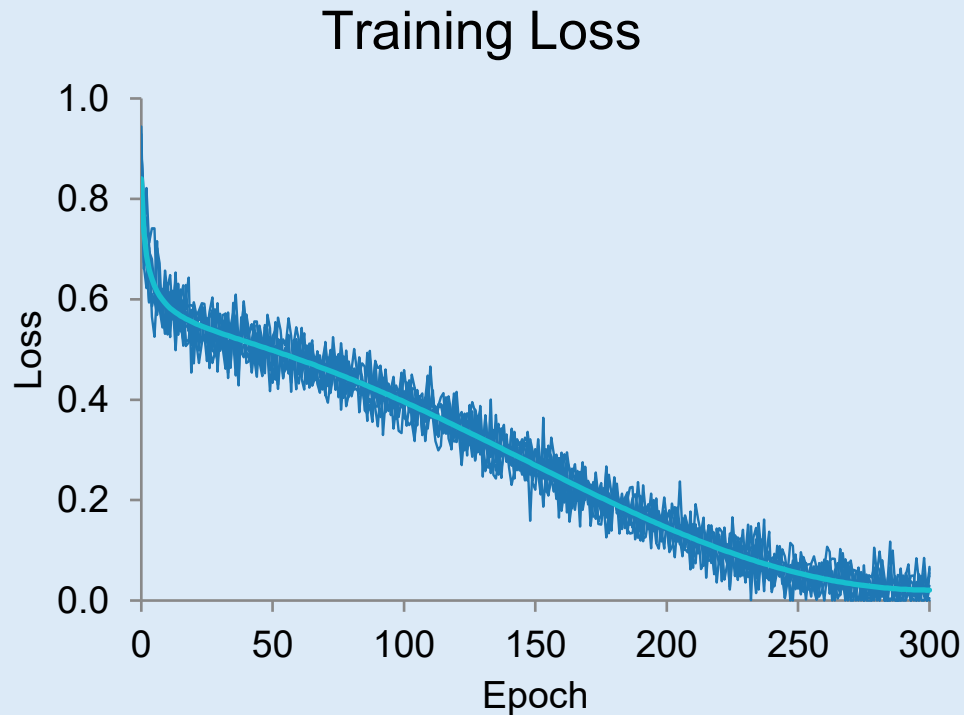
α_t : Learning rate at epoch t

T : Total number of epochs

Learning rate decay

Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine:
$$\alpha_t = \frac{1}{2}\alpha_0 \left(1 + \cos \left(\frac{t\pi}{T} \right) \right)$$

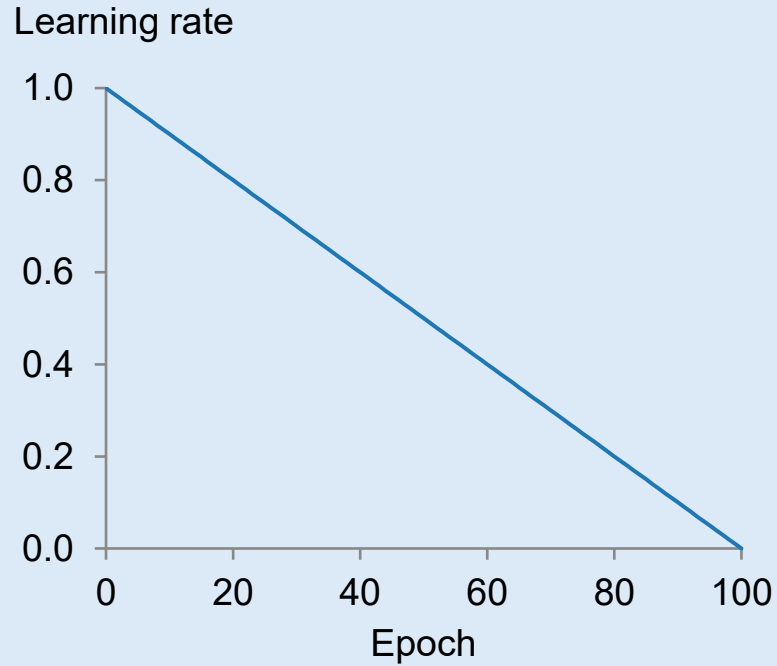


α_0 : Initial learning rate

α_t : Learning rate at epoch t

T : Total number of epochs

Learning rate decay



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine:
$$\alpha_t = \frac{1}{2}\alpha_0 \left(1 + \cos \left(\frac{t\pi}{T} \right) \right)$$

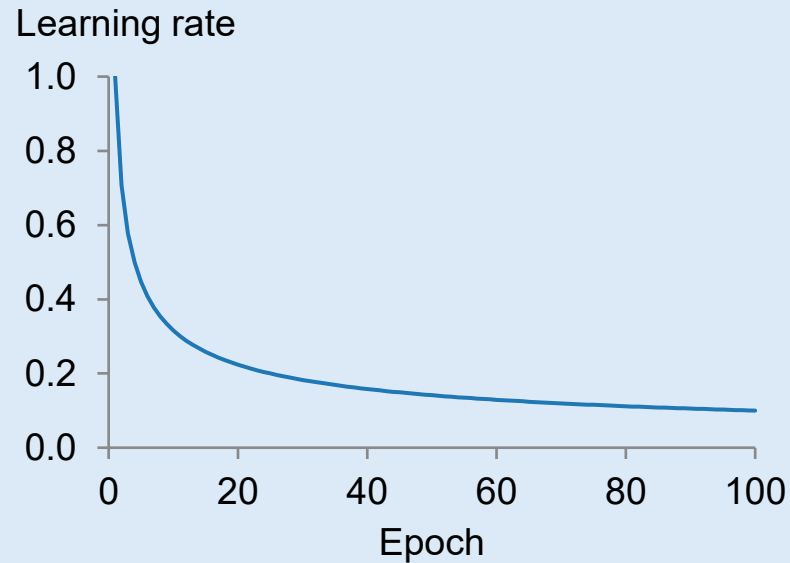
Linear:
$$\alpha_t = \alpha_0 \left(1 - \frac{t}{T} \right)$$

α_0 : Initial learning rate

α_t : Learning rate at epoch t

T : Total number of epochs

Learning rate decay



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine: $\alpha_t = \frac{1}{2}\alpha_0 \left(1 + \cos \left(\frac{t\pi}{T} \right) \right)$

Linear: $\alpha_t = \alpha_0 \left(1 - \frac{t}{T} \right)$

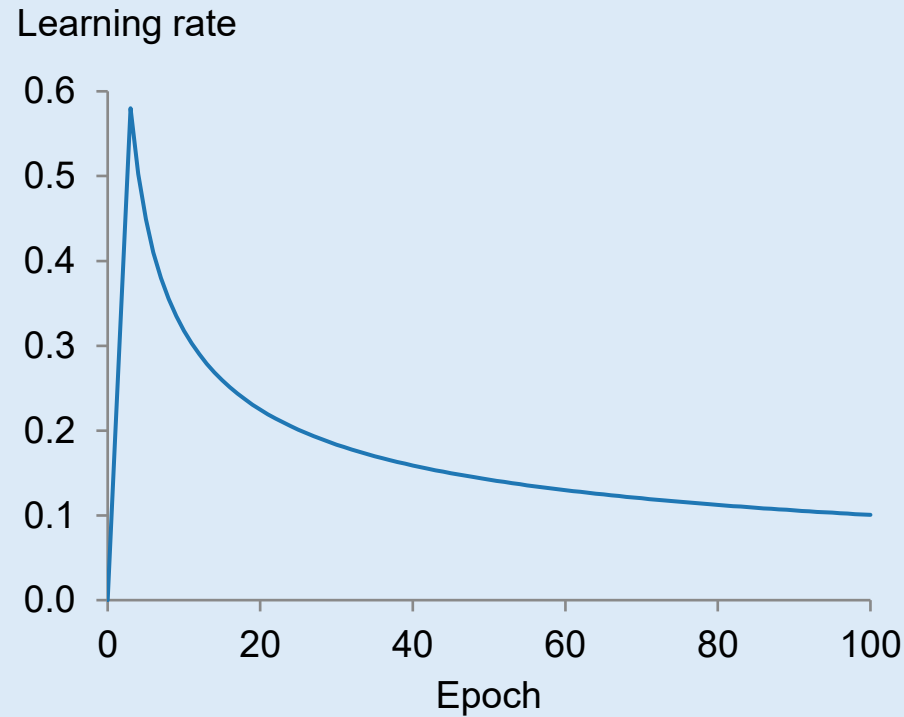
Inverse sqrt: $\alpha_t = \frac{\alpha_0}{\sqrt{t}}$

α_0 : Initial learning rate

α_t : Learning rate at epoch t

T : Total number of epochs

Learning rate decay: linear warmup



High initial learning rates can make loss explode; linearly increasing learning rate from 0 over the first ~5,000 iterations can prevent this.

Empirical rule of thumb: If you increase the batch size by N , also scale the initial learning rate by N

In practice:

- **Adam** is a good default choice in many cases; it often works ok even with constant learning rate.
- **SGD+Momentum** can outperform Adam but may require more **tuning of LR and schedule**.

Next story: How to optimize more complex functions?

(Currently) Linear score function: $f = Wx$

$$x \in \mathbb{R}^D, W \in \mathbb{R}^{C \times D}$$

Neural networks: 2 layers

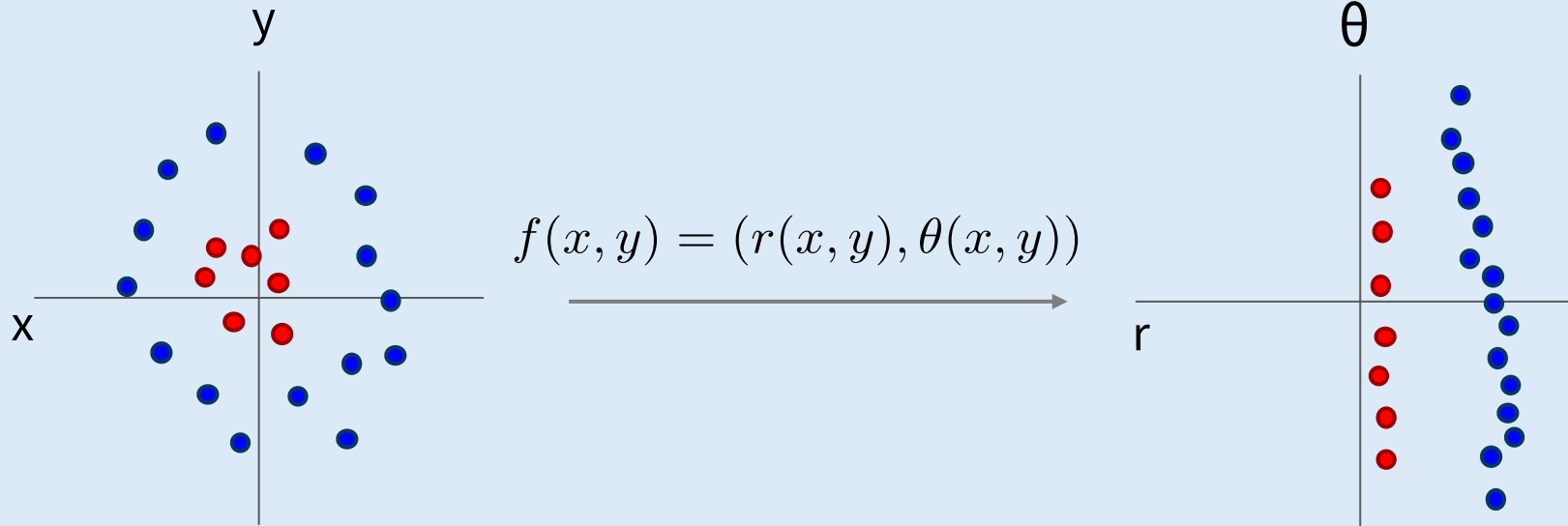
(Currently) Linear score function: $f = Wx$

(Next Class) 2-layer Neural Network: $f = W_2 \max(0, W_1 x)$

$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

(In practice we will usually add a learnable bias at each layer as well)

Why do we want non-linearity?



Cannot separate red and blue points with linear classifier.

After applying feature transform, points can be separated by linear classifier.

Neural networks: also called fully connected network

(Currently) Linear score function: $f = Wx$

(Next Class) 2-layer Neural Network: $f = W_2 \max(0, W_1 x)$

$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

“Neural Network” is a very broad term; these are more accurately called “fully-connected networks” or sometimes “multi-layer perceptrons” (MLP)

(In practice we will usually add a learnable bias at each layer as well)



References

- Stanford CS231n
- Mathematical Foundations of Deep Neural Networks at Seoul National University.

